GUÍA INTEGRAL DEL ESTUDIANTE

ACTIVIDAD 4 · TECNOLOGÍA WEB II · LABORATORIOS 12, 13 Y 14

Se deben implementar servicios de intercambio de datos, API REST e interfaz AJAX; verificar funcionamiento y entregar informe técnico APA 7, bitácora de IA, documentación y repositorio GitHub.

1. Qué se debe entregar

Producto	Contenido obligatorio
Código	Laboratorios 12, 13 y 14 completos, basados en las carpetas inicio
API documentada	Cada endpoint con método, ruta, autenticación, petición, respuesta, errores y evidencia
Informe técnico	Documento final APA 7 con resultados y capturas propias
Bitácora de IA	Interacciones reales, decisiones, correcciones y verificación en los tres laboratorios
GitHub	Repositorio con README, evidencias, informe y presentación .pptx, .html o .pdf

2. Orden recomendado para avanzar sin perder tiempo

Trabajar en secuencia 12 → 13 → 14. Antes de cada laboratorio ejecutar su verificador inicial, completar sus TODO y volver a verificar. No copiar carpetas «solucion» ni publicar los solucionarios docentes. Reutilizar la base de datos y los componentes anteriores de la asociación.

3. Preparación de Windows y PowerShell

Paso 1. Comprobar herramientas

Abrir PowerShell. Ejecutar los comandos uno a uno. Si alguna herramienta no existe, comunicarlo al docente antes de continuar.

```
php -v
composer --version
docker --version
docker compose version
git --version
php -m
```

Verificar las extensiones curl, dom, xml, xmlwriter, pdo_mysql y soap (si se utiliza SOAP). La presencia de Docker no significa que todos los servicios estén iniciados.

Paso 2. Preparar directorios y revisar Docker

Adaptar la ruta a la carpeta donde se descargaron los laboratorios. Evitar iniciar dos servidores en el mismo puerto.

```
New-Item -ItemType Directory -Force C:\TW2\actividad4\evidencias
docker compose config --services
docker compose ps
```

Los comandos docker compose se ejecutan dentro de la carpeta que contiene compose.yaml. Si el proyecto usa PHP local, ejecutar Composer y PHP en PowerShell; si usa contenedor, hacerlo dentro del servicio PHP real identificado por docker compose config --services.

Paso 3. Verificar el proyecto de inicio

En la carpeta inicio del laboratorio actual, ejecutar:

```
composer dump-autoload
php verificar.php
```

Guardar una captura del resultado inicial. Para servir una aplicación PHP cuya estructura contiene public/index.php:

```
php -S 127.0.0.1:8000 -t public public/index.php
```

Dejar esa consola abierta; usar una segunda para pruebas. Si el entorno oficial utiliza Docker, seguir su archivo de instalación en vez de mezclar métodos.

4. Laboratorio 12 — Intercambio JSON, XML, RSS y SOAP/REST

Paso 4. Generar catálogo JSON

Abrir /datos/catalogo.json y comprobar Content-Type application/json, ISO 8601 para fechas y precios decimales como texto (ejemplo «45.00»). Completar los TODO de la carpeta inicio.

```
curl.exe -i http://127.0.0.1:8000/datos/catalogo.json
```

Paso 5. Generar XML correctamente

Publicar /datos/catalogo.xml con XMLWriter. Comprobar que «Polen & propóleo» se represente como «Polen & propóleo». No concatenar texto sin escape.

```
curl.exe -i http://127.0.0.1:8000/datos/catalogo.xml
```

Paso 6. Validar pedidos con XSD y JSON

Probar un pedido correcto y otro con fecha inválida, código fuera del patrón o cantidad negativa. Para XML usar DOMDocument con LIBXML_NONET y schemaValidate; para JSON verificar tipos, estructura, fecha y rangos. Explicar por qué se rechaza cada dato.

Paso 7. Generar RSS

Comprobar /noticias.rss, canal RSS 2.0 con title, description, guid y pubDate compatible con DATE_RSS. Este mismo recurso será leído desde AJAX en el Laboratorio 14.

```
curl.exe -i http://127.0.0.1:8000/noticias.rss
```

Paso 8. Comparar SOAP y REST

Consultar el mismo producto con el cliente SOAP del laboratorio y con GET /datos/productos/MIE-500. Registrar método, formato, respuesta, errores y bytes medidos en el entorno del estudiante. No copiar cifras de ejemplo como si fueran mediciones propias.

Paso 9. Consumir un servicio público desde PHP con cURL

Crear consumir_publico.php como demostración adicional de la consigna, comprobar HTTP y decodificar JSON. La ejecución requiere Internet.

```
<?php
$ch = curl_init('https://jsonplaceholder.typicode.com/todos/1');
curl_setopt_array($ch, [CURLOPT_RETURNTRANSFER=>true, CURLOPT_TIMEOUT=>10,
  CURLOPT_HTTPHEADER=>['Accept: application/json']]);
$body = curl_exec($ch);
if ($body === false) { throw new RuntimeException(curl_error($ch)); }
$status = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);
if ($status !== 200) { throw new RuntimeException("HTTP $status"); }
print_r(json_decode($body, true, 512, JSON_THROW_ON_ERROR));
php consumir_publico.php
php verificar.php
```

Evidencias del Lab 12: JSON, XML escapado, validación correcta/incorrecta, RSS, tabla SOAP/REST con mediciones, consumo cURL y verificador. Referencia de la guía: 15/15 en la solución docente; el estudiante debe demostrar su propio resultado.

5. Laboratorio 13 — API REST

Paso 10. Identificar el contrato

Ruta base http://127.0.0.1:8000/api/v1. Revisar la carpeta inicio y documentar todos los endpoints efectivamente implementados. Las rutas fijas como /productos/sugerencias deben preceder a /productos/{codigo}.

Método	Ruta	Resultado
GET	/productos	200: lista con datos, meta y enlaces
GET	/productos/{codigo}	200 / 304 / 404
POST	/productos	201 + Location
PUT	/productos/{codigo}	200
DELETE	/productos/{codigo}	204 sin cuerpo
GET	/productos/sugerencias?q=jab	200 / 422
OPTIONS	/productos	204: preflight CORS

Paso 11. Consultar catálogo y paginación

Solicitar una lista con filtros y revisar datos, meta y enlaces. Un parámetro inválido debe generar 422, no corregirse silenciosamente.

```
curl.exe -i "http://127.0.0.1:8000/api/v1/productos?pagina=1&por_pagina=5"
```

Paso 12. Crear un producto con token

Crear producto.json con campos reales del contrato y socio_id existente. Obtener un token autorizado de la herramienta de laboratorio; nunca guardarlo en GitHub.

```
{
  "codigo":"MIE-ROS", "nombre":"Miel de romero",
  "categoria":"Miel", "precio":"50.00", "socio_id":1
}
$TOKEN = Read-Host "Token API"
```

```
curl.exe -i -X POST "http://127.0.0.1:8000/api/v1/productos" `
 -H "Authorization: Bearer $TOKEN" `
 -H "Content-Type: application/json" --data-binary "@producto.json"
```

Resultado esperado si los datos son válidos y el rol está autorizado: 201 Created, cabecera Location. El archivo debe guardarse en la carpeta desde donde se ejecuta curl.exe.

Paso 13. Actualizar y eliminar

Crear actualizacion.json con los campos requeridos. Ejecutar PUT y DELETE sobre un producto de prueba; después confirmar con GET que fue eliminado.

```
curl.exe -i -X PUT "http://127.0.0.1:8000/api/v1/productos/MIE-ROS" `
 -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" `
 --data-binary "@actualizacion.json"
curl.exe -i -X DELETE "http://127.0.0.1:8000/api/v1/productos/MIE-ROS" `
 -H "Authorization: Bearer $TOKEN"
```

Paso 14. Probar errores y seguridad

Demostrar 400 JSON roto, 401 token ausente, 403 rol sin permiso, 404 recurso ausente, 405 método inválido, 415 Content-Type incorrecto, 422 datos inválidos. Usar problem+json; comprobar que los tokens se guardan como hash SHA-256 y que OPTIONS devuelve cabeceras CORS.

Paso 15. Documentar con OpenAPI 3.1

Registrar rutas, parámetros, esquemas, seguridad Bearer, ejemplos y códigos HTTP. Validar el archivo OpenAPI en un editor compatible. Revisar ETag / If-None-Match y 304 sin cuerpo.

```
php verificar.php
```

Evidencias del Lab 13: capturas GET, POST 201 + Location, PUT, DELETE 204, 401/403, 422 problem+json, ETag 304, OPTIONS, OpenAPI y verificador. Referencia de la solución docente: 15/15.

6. Laboratorio 14 — AJAX moderno con fetch

Paso 16. Preparar navegador y verificador

Abrir /catalogo en Chrome o Edge y DevTools (F12 → Red → Fetch/XHR). Ejecutar npm install solo si se utilizarán las pruebas de navegador opcionales; después php verificar.php.

```
composer dump-autoload
npm install
php verificar.php
```

Paso 17. Dibujar la grilla sin XSS

En public/js/catalogo.js sustituir innerHTML con datos por document.createElement, textContent y replaceChildren. Comprobar que Miel aparezca como texto literal.

```
const tr = document.createElement('tr');
const td = document.createElement('td');
td.textContent = producto.nombre;
tr.append(td);
tabla.append(tr);
```

Paso 18. Centralizar fetch y errores

En public/js/api.js comprobar response.ok; fetch no rechaza automáticamente respuestas 404 o 500. Leer problem+json y mostrar aviso accesible role=alert. Distinguir fallos de red y AbortError.

```
const respuesta = await fetch(ruta, {headers:{Accept:'application/json'}, signal});
if (!respuesta.ok) {
  const problema = await respuesta.json();
  throw new Error(problema.detail || `HTTP ${respuesta.status}`);
}
const datos = await respuesta.json();
```

Paso 19. Cancelar solicitudes viejas

Usar AbortController en cada carga de grilla y debounce de 250 ms en la búsqueda. Probar con red lenta y búsquedas «mi», «mie», «cera»; confirmar (canceled) en DevTools y que la última búsqueda prevalezca.

```
let controlador = null;
async function cargar(ruta) {
  controlador?.abort();
  controlador = new AbortController();
  try { return await obtener(ruta, controlador.signal); }
  catch (e) { if (e.name === 'AbortError') return; throw e; }
}
```

Paso 20. Autocompletar accesible

Consultar /api/v1/productos/sugerencias?q=jab. Implementar role=combobox, listbox, option, aria-expanded, aria-activedescendant y aria-selected; flechas arriba/abajo, Enter y Escape. Cerrar la lista debe reiniciar la selección activa.

Paso 21. Validación en vivo y RSS

Para código abc: aviso local sin petición. Para MIE-500: GET 200 significa ocupado; POL-999: GET 404 significa disponible. La validación del servidor permanece obligatoria. Leer /noticias.rss con DOMParser, verificar parsererror y mostrar cinco novedades con textContent.

```
const r = await fetch('/noticias.rss');
if (!r.ok) throw new Error(`HTTP ${r.status}`);
const xml = new DOMParser().parseFromString(await r.text(),'application/xml');
if (xml.querySelector('parsererror')) throw new Error('RSS inválido');
const items = [...xml.querySelectorAll('item')].slice(0,5);
```

Paso 22. Verificación final

Probar paginación, orden, autocompletar con teclado, cancelaciones, error 500, RSS y validación en vivo. Ejecutar php verificar.php y guardar capturas. Referencia: 18/18 con navegador o 8/8 de servidor más pruebas manuales cuando no se dispone de Node.

7. Ficha obligatoria de documentación de cada endpoint

Repetir esta ficha por cada ruta, incluidos los recursos del Laboratorio 12. Sustituir ejemplos por datos obtenidos en pruebas reales.

Campo	Ejemplo
Nombre	Consultar producto
Método y URL	GET /api/v1/productos/MIE-500
Autenticación	Lectura pública según contrato
Petición	Accept: application/json
Respuesta 200	{"codigo":"MIE-500","nombre":"Miel multifloral 500 g","precio":"45.00","stock":30}

Errores	404 no encontrado; 304 con ETag válido
Evidencia	Captura con URL, cabeceras, status y cuerpo

8. Matriz de evidencias que debe completar el estudiante

Código	Evidencia	Estado
E01	Versiones y entorno	Pendiente
E02	Catálogo JSON	Pendiente
E03	XML y escape	Pendiente
E04	Validación XSD/JSON	Pendiente
E05	RSS	Pendiente
E06	SOAP/REST y cURL	Pendiente
E07	GET paginado	Pendiente
E08	POST 201	Pendiente
E09	PUT 200	Pendiente
E10	DELETE 204	Pendiente
E11	Errores 401/403/422	Pendiente
E12	OpenAPI/ETag/CORS	Pendiente
E13	Grilla AJAX	Pendiente
E14	Paginación y orden	Pendiente
E15	Autocompletar accesible	Pendiente
E16	Cancelaciones en DevTools	Pendiente
E17	Validación en vivo	Pendiente
E18	RSS en interfaz	Pendiente
E19	Verificadores finales	Pendiente
E20	GitHub e informe	Pendiente

9. Informe técnico final — formato APA 7

Estructura: portada académica; resumen (150–250 palabras); introducción; objetivos; fundamentos JSON/XML/REST/SOAP/AJAX; metodología y herramientas; implementación Lab 12; implementación Lab 13; implementación Lab 14; documentación completa de endpoints; pruebas y evidencias numeradas; bitácora de IA; resultados y discusión; conclusiones; referencias; anexos con URL de GitHub.

Presentación APA 7: márgenes 2,54 cm, interlineado doble, paginación, títulos jerárquicos y referencias con sangría francesa. Utilizar una fuente admitida (p. ej. Times New Roman 12 o Arial 11). Cada figura debe identificarse y comentarse; citar las fuentes consultadas y no inventar referencias.

10. Bitácora de IA para toda la actividad

Fecha/Lab	Prompt real	Propuesta IA	Decisión propia	Prueba
__/__/__ · 12	Cómo generar XML seguro	Respuesta resumida	Acepté/corregí/rechacé y por qué	XML validado
__/__/__ · 13	Cómo manejar errores HTTP	Respuesta resumida	Ajuste a problem+json	Prueba 422
__/__/__ · 14	Cómo evitar	Respuesta	AbortController y	DevTools

	carreras fetch	resumida	debounce	(canceled)

Registrar todas las consultas relevantes con modelo y fecha, explicación crítica, cambios realizados y verificación. Estas filas son una plantilla: reemplazarlas por interacciones reales. Respetar las restricciones de uso de IA indicadas por el docente y no usar IA durante una validación que la prohíba.

11. Repositorio GitHub y presentación

```
actividad4-tecnologia-web-ii/
  README.md
  lab12/  lab13/  lab14/
  docs/endpoints.md  docs/openapi.yaml  docs/BITACORA_IA.md
  evidencias/
  informe/Informe_Actividad4.pdf
  presentacion/Actividad4.pptx
```

Crear .gitignore antes de subir: excluir .env, claves, tokens, credenciales, vendor/, node_modules/ y datos privados. El README debe explicar instalación, requisitos, ejecución, endpoints y pruebas. La presentación puede ser .pptx, .html o .pdf e incluir arquitectura, demostración, resultados y conclusiones.

```
git init
git add .
git commit -m "docs: entrega actividad 4"
git branch -M main
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git push -u origin main
```

Crear previamente el repositorio remoto y sustituir USUARIO/REPOSITORIO. Comprobar que el enlace sea accesible y aparezca tanto en el informe como en el README.

12. Lista final antes de entregar

☐ Laboratorio 12 verificado y documentado

☐ Laboratorio 13: CRUD, autenticación y errores comprobados

☐ Laboratorio 14: fetch, grilla, autocompletar, RSS y validación probados

☐ Capturas propias con resultados reales

☐ Todas las rutas documentadas con peticiones y respuestas

☐ Informe técnico final con estructura APA 7

☐ Bitácora de IA completa de los tres laboratorios

☐ Repositorio GitHub funcional y presentación incluida

☐ Sin secretos, tokens ni solucionarios docentes publicados

13. Criterios orientativos de valoración

Componente	Peso
Lab 12: intercambio de datos	30 %
Lab 13: API REST	40 %
Lab 14: AJAX	30 %

Informe, evidencias, bitácora y repositorio se verifican transversalmente. Los resultados 15/15, 15/15 y 18/18 son referencias de los solucionarios docentes, no resultados garantizados de cada estudiante.

FIN DE LA GUÍA · Entregar una implementación funcional y demostrable, no únicamente documentación.